

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 314 — Past Questions & Materials
by Renance

Course: CSC 314
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 6
Levels: 300

This study pack was compiled and restructured by Renance for CSC 314 students at Federal University of Petroleum Resources, Effurun. It brings together 6 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 6 — CSC 314 Object Oriented Programming 2024/2025.

Level 300 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN
DEPARTMENT OF COMPUTER SCIENCE
1ST SEMESTER UNDERGRADUATE EXAMINATION (2024/2025 SESSION)
CSC 314: OBJECT-ORIENTED PROGRAMMING

INSTRUCTIONS:

1. Attempt Any Four (4) Questions. One (1) Compulsory.
2. Time Allowed: 2 ½ Hours.
3. Answer Each Question On A New Page.

Date: 2nd May, 2025**Question 1**

- a) i). Define Objected oriented programming? (3 marks)
 ii). State and explain the principle of Object-Oriented Programming. (4 marks)
 iii). List the benefit of the object-oriented programming. (4 marks)
- b) i). Give the brief history of java programming? (2 marks)
 ii). List the resources need to write a Java program. (3 marks)
- c) i). What is translator? (2 marks)
 ii). List and explain the types of translators. (2 marks)

Question 2

- a) i). Define Program Flow Chart? (3 marks)
 ii). Mention all the symbols and their descriptions used in Program Flow Chart. (6 marks)
- b) List and explain the Program Constructs in Program Flow Chart. (6 marks)
- c) Connectors are sometimes use in drawing a flowchart. Explain when a connector should be used. (2 marks)
- d) Using the flowchart diagram, design a program. If a program is a menu driven program, it allows the user to make a choice. If the user selects the first option, it displays a message "option one selected". If option two is selected, it displays a message "option two selected" and if user selects option three, it displays a message "option three selected". Lastly, if the user entered a wrong option, a message "invalid option" will be displayed. (3 marks)

Question 3

- a) Define the following term: (1 mark each)
 i). Operators ii). Separators iii). Identifiers iv). keywords
- b) i). List all the types of operators used in Java programming. (6 marks)
 ii). List all the primitives and reference Data types (4 marks)
- c) In a tabular form list the six (6) separators and their functions used in Java programming. (6 marks)

Question 4.

- a) In an Object-Oriented Programming (OOP) language, what is an "exception", give any three (3) examples? (5 marks)
- b) Explain the terms:
 i). Exception object (1 mark)
 ii). Custom Exceptions (1 mark)
 iii). Give three (3) benefits of using Exceptions in OOP language. (3 marks)
- c) What are the five main methods of applet? (5 marks)
- d) What is an Abstract class? List the 4 basics of OOP abstraction? (5 marks)

Question 5.

- a) Explain the following terms: (2 marks each)
 i). Applet ii). Abstraction iii). Object linking and embedding (OLE) iv). Persistence v). Window tool-kit
- b) What is an immutable object? Give 4 examples of immutable objects. (5 marks)
- c) Explain the term "Object collection", and give one (1) characteristic each for "ArrayList", "Linked Lists" and "multi-dimensional arrays". (5 marks)

Paper 2 of 6 — CSC 314 2018-2019

Level 300 • First Semester • PastQuestion

1' , ~

l~)

-FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

COLLEGE OJ; SCIENCE

DEPARTMENT OF MATHEMATICS, COMPUTER SCIENCE

FIRST SEMESTER EXAMINATION 2018/2019 SESSION

CSC 314: Object Oriented Programming Time: 2 ½ hours

Instructions: Question 1 and any other three.

Answers:

QUESTION 1 (Compulsory) _____

(a) The area of a circle is given by πr^2 where r is the radius of the circle. The area of a rectangle is given by Length $\times$ Breadth. The surface area of a cylinder of height h is the addition of the areas of the circles at the ends of the cylinder and the area of the rectangle whose Length is the height of the cylinder and breadth is the perimeter of the circle at the end of the cylinder. Thus the surface area of a cylinder is given by the formula: $2\pi r^2 + 2\pi rh$.

i) Write a good Java program with a class named Circle. having two methods that compute the area and the circumference of the circle respectively.

ii) Write another Java class named Rectangle having one method for computing the area of a rectangle.

iii) Reuse the Circle and Rectangle classes in a main class named Geometry:Area to calculate and display the surface area of a cylinder. This class should also display the area of the circle at the end of the cylinder. Hint: the user's inputs should be read from the keyboard.

(/ 5 marks)

(b) Define the following object oriented programming concepts and extract Java code snippets from your solution to QUESTION 1a to illustrate them with examples: ~,

object, method, instantiation, inheritance, polymorphism and constructors.

(10 marks)

QUESTION 2

(a) Declare a multi-dimensional array using Java. (2 marks)

(b) A is an m by n matrix and B is a p by q matrix. Write a good Java program to multiply matrices A and B and compute the solution into matrix C . Your program should test matrix A and B for matrix multiplication compatibility.

(/ 3 marks)

QUESTION 3

(a) Why is ArrayList better than Array? . (2 marks)

ii) Write Java snippets to create an ArrayList, add elements to the ArrayList and remove elements from it. (5 marks)

(b) Write a good Java program to compute the average score of 10 subjects. (8 marks)

1

```
(1) \ , , 11 . 111 . r l q 11 \ n l l 1 1 d i s p l : 1 > l h l . \ \ \ l i \ \ i 11 g i n f u r t 11 i l t i u 11 111 a \ \ l ' h b t l ) \ \ ~ 1 : 1
l " 111 g J a v a
```

```
., p p k t ~ .
```

```
; -- : \ i \ \ . . l l t h l l l ) d i
```

```
P U ' . \ " r i \ t L N r : C l l l l 11 l l l l l ' l ~ l . i r n , : e
```

```
~ l : \ l ' l U L ' . N U . : C t ) ~ l ( ) l l ) l
```

li) l' l r n l ~ . 1111 t l i n r t h e , t l . p s ~ - l \ l \ \ ' i 111 l j t : i ~ r l (' n r u l l ' a n j r u 11 a n : i p p k l t u d i s p l a y

the: i n l t 1111 r n t i 11 n (j 1111 l r j . _ 1)

(ii) \ \ r i t e : . 111 : q , p k l : 1 n j . 111 h t i n l j p , ' l l l l l ' l l l l l d i s p l u ~ . 1 h r i 11 l i l r r n : i l i P 11 . (8 m a r k s)

tb) ti) \ \ ' 111 r j a , a , 111 k : - n i p p r t . s t i , r \ \ \ 1 111 r t h 11 d . s 11 f t h r , . \ u j i , , C l i p i n 1 n l : 1 c r i 11 _ j a v a . a p p l c : 1

p . i , b g l . (3 m a r k s)

l l l \ 11 , l i ~ 11 h ~ . i l l 1111 . 1 ~ , 1 \ 11 i i 11 d l l ' . 1 p p i l ' 11 \ l i i d 1 n 11 . 1 l 111 j \ i l l : , - 0 11 l b \ . ' a n d i n w h i c h c l u ~ s

i , 111 . 111 r 1 h , , d f p u r i . l ' . (? m a r k . 1)

```
" - H ' l ' ~ l ' l l ) ~ : -
```

```
( . 1 ) l ; , \ e n 1 h 1 . i " , 111 , \ , i n ~ j . 1 \ : 1 s t : 11 , 1 n r 111 :
```

```
~ t r i l l ~ k t t a s " l h : \ l ' 111 : 1 s t i : - r , j ~ l r i n g m n n i p 11 l a l i o n i n . l : 1 v a " :
```

```
\ \ r i t . j a , a s t : 11 , m . n t l . - ) 11 , d i s p i n ~ . t h e ~ h ' l ' r n r u c t l . ' r i n t h e ~ l r i n g a n d \ \ t i l e
```

```
l h l ' 1 ' u t r 111 l i f ~ l l l l r , - , j r . ( 3 l l / O r b )
```

```
1 t - - ) l 11 \ l ' 1 . l l i j , ' \ j m p k : - , , r t l w ~ , n a a l s ~ 111 a x , , f t h t l ( 1 l l o w i n g c o n t r o l l c m l o n s t r u c t s
```

```
111 . l . 1 . 1 l ! ' - t h , 11 - 1 . l ~ , . F N - L , , , , p . 1 d l i k L o o p . d o - w h i l e L o , , p . r S 111 u r k . 1 j
```

```
l \ ~ t u j ~ l h t . ( , , 11 , , " i n g . l . l \ : i 1 . l d c :
```

```
S t r i n g m ~ s - 1 p h q 1 c s { l i z ~ & S i . I A , m 9 o 3 v H i Y e t l % 1 " ;
```

```
S t r i n g d e c 0 d e d =
```

```
r o r ( l n t 1 = 0 : , , m ) g . l e n g t h ( ) . 1 - t - t )
```

```
d ( 1 . . . . _ . : == L 1 \
```

```
0 e C 1 . ' 0 t ' d • = m ~ i ; ( h , 1 r . \ q 1 )
```

```
S p t e m O u t p r , n t l n ( d u , , d e d m t > s ~ j t ; e ' + d e c o d e d ) ,
```

```
, \ r , t e : r i e o u t p u t c ~ t h e , 0 d e ( . / 11 / U r k . 1 )
```

```
~ • l l ' . ' l l U \ l l
```

```
l 5 111 , i r k s )
```

```
( t ' l \ \ , 1 e . 1 l j j r r • ~ r j m t h j t 11 t l l j l l l 111 . 1 u , a h ' ~ l ' k c t , i - , , 111 t h e l ( 1 l l o w i 11 ~ t h r t : ~
```

```
_ p t 11 . , n , c t . l l l \ T \ l m j d l l n t l \ , 1 \ 111 ~ , \ 1 . , , p u n t . 2 . C u r r , m A c c l , u n t . _ , C : m c d .
```

```
r ,
```

```
I t t h e u , r r , r l r , , , , \ j l j j . i . u u n i 11 p l ' t h . i t i ~ ~ . , \ 111 ! , : - l l r e u r r t . n t . t h l . \ l 111 . 11 : h i n l '
```

```
, n , • u l j r r , • m r t t h , u , , , 1 \ i t h . j r . 11 \ , d . • t l i l 1 \ t h l ' l l ~ t " r h ) \ \ i t h d r . 11 \ \ : - . ' . 1 i r : t f r , , 111 . l f l
```

111111.11 J,u.lUllt b.JJJ1u: t il l NoL !. 1.,•111p111l• lhl' ,K' c' lIUOI b:.tlanc't' .111J dispb~ th is
t-.11Jn,e tl• the u,er .1 f1cr the l\ i1hJr.1\1 JI //(l 11111rk..J

Paper 3 of 6 — CSC 314 Object-Oriented Programming: (3 Units) Course Handout

Level 300 • First Semester • Handout

CSC 314 Object-Oriented Programming: (3 Units)

A MUSTAPHER

Review of CSC121 concepts: Procedural programming, iterative development of “growing” algorithms, parallel arrays. File input and output: Character-oriented file I/O, line-oriented file I/O. Basic OOP Concepts: Classes, Objects, inheritance, polymorphism, Data Abstraction, Tools for developing, Compiling, interpreting and debugging, Java Programs, Java Syntax and data objects, operators. Control flow constructs, objects and classes programming, Arrays, methods.

B STAR

Exceptions, Applets and the Abstract, OLE, Persistence, Window Toolkit, Laboratory exercises in an OOP Language. Strings: Character manipulation, string manipulation, immutable objects. Object Collections: The ArrayList class, ArrayList operations. Lists. Organization of linked lists, linked lists using a Node class. Multi-dimensional arrays.

EXCEPTIONS

In an Object-Oriented Programming (OOP) language, an "exception" is a mechanism used to signal an unexpected error or abnormal condition that occurs during program execution, allowing the program to gracefully handle the issue instead of crashing; it is represented as an object that can be caught and dealt with using specific code blocks, enabling centralized error handling and improved code readability compared to traditional error codes.

Key points about exceptions in OOP:

□ Exception object:

When an error occurs, the program throws an exception object, which contains information about the error type and details about the situation.

□ Try-Catch blocks:

Code that might potentially throw an exception is placed within a "try" block, and the code to handle the exception (if thrown) is placed within a corresponding "catch" block.

□ Exception hierarchy:

Most OOP languages have a hierarchy of exception classes, allowing for specific handling of different types of errors.

□ Custom exceptions:

Developers can create their own exception classes to represent application-specific errors, providing more context and clarity for error handling.

Common examples of exceptions in OOP languages:

- ZeroDivisionError: Attempting to divide by zero.
- IndexError: Accessing an index outside the bounds of a list or array.
- FileNotFoundError: Trying to open a file that doesn't exist.

□ `TypeError`: Trying to use a value of the wrong data type.

□ `NullPointerException`: Accessing a null reference as if it were a valid object.

Benefits of using exceptions in OOP:

□ Improved code readability:

Separates error handling logic from the main program flow, making code easier to understand.

□ Flexibility:

Allows for different error handling strategies depending on the type of exception thrown.

□ Error propagation:

Exceptions can be propagated up the call stack, allowing higher-level functions to handle errors that occur in lower-level functions.

APPLET

An applet is a small program that is intended not to be run on its own, but rather to be embedded inside another application. The `Applet` class must be the superclass of any applet that is to be embedded in a Web page or viewed by the Java Applet Viewer. An applet is a small computer program that performs a specific task. It is typically embedded within another larger app or software platform and has limited functionality. This allows applets to run quickly and reliably without demanding a lot of system resources.

What are the two types of applets?

In Java, there are two types of Applet

□ Java Applets based on the AWT(Abstract Window Toolkit) packages by extending its `Applet` class.

□ Java Applets is based on the Swing package by extending its `JApplet` Class in it.

Local applet

This is a type of applet specific to Java applets. Local applets are created and saved on the local system. When an HTML file contains a local applet, a web page does not need to get data from the internet. The filename or pathname within the code specifies or defines it.

What are the five main methods of applet?

There are five main methods in the applet life cycle. These are: `init()`, `start()`, `stop()`, `paint()` and `destroy()`. These methods are invoked for execution. An applet is also a class Java which is created by extending the class java.

Abstract class

Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class). Abstract method: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

What are the 4 basics of OOP abstraction?

OOP allows objects to interact with each other using four basic principles: encapsulation, inheritance, polymorphism, and abstraction. These four OOP principles enable objects to communicate and collaborate to create powerful applications

Example of an abstract data type in OOP

ADT is a container which holds different types of objects with specifications. logical representation(i.e an interface or protocol) of the data and the operations to manipulate the component elements of the data. Examples of ADT: List, Map, Set, Stack, Queue, Tree, Graph

The abstract keyword in Java is used to declare a class or a method that cannot be instantiated directly or must be implemented by subclasses, respectively. It is a key part of Java's abstraction mechanism, allowing developers to define abstract classes and methods that provide a blueprint for other classes.

An example of abstraction in OOPS is the use of abstract classes and methods. Abstract classes serve as templates or blueprints for objects but cannot be instantiated on their own. Abstract methods defined in these classes lack implementation details and must be provided by concrete (non-abstract) subclasses.

Abstraction is the progress of modelling real-world objects into programming language. Abstraction is not about interfaces or abstract classes.

Hence interfaces and abstract classes are just two techniques used in this progress.

In an Object-Oriented Programming language like Java, everything is an abstraction: interface, class, field, method, variable, etc.

Abstraction is the fundamental concept on which other concepts rely: encapsulation, inheritance, and polymorphism

Definition 2: Abstraction is one of the key concepts of object-oriented programming (OOP) languages. Its main goal is to handle complexity by hiding unnecessary details from the user. That enables the user to implement more complex logic on top of the provided abstraction without understanding or even thinking about all the hidden complexity.

OLE Object Linking and Embedding

In the context of Object Oriented Programming (OOP), "OLE" stands for "Object Linking and Embedding," which refers to a Microsoft technology that allows users to embed objects created in one application within another application, essentially letting you integrate data from different programs into a single document; for example, inserting an Excel spreadsheet into a Word document.

Object Linking and Embedding (OLE) is a proprietary technology developed by Microsoft that allows embedding and linking to documents and other objects.

Key points about OLE:

□ Functionality:

OLE enables users to "link" or "embed" objects, meaning the data can either be dynamically updated from the source application when changes are made, or be copied directly into the container application as a static element.

□ Implementation:

Within an OOP language, OLE functionality is typically achieved through specific libraries or APIs provided by the operating system, allowing developers to create applications that

can interact with objects from other applications.

□ Example scenario:

Imagine creating a presentation in PowerPoint where you want to embed a live graph from Excel. Using OLE, you could insert the Excel chart directly into the slide, and any changes made to the Excel source file would be reflected in the presentation.

OLE (Object Linking and Embedding) objects are active links in your project to documents created and edited in other applications. This enables you to insert formatted objects that you cannot create directly in the Help+Manual editor, such as formulas created in the MS Equation Editor.

OLE operation

Object Linking and Embedding (OLE) is a feature that allows Office applications to interact with other applications. Common reasons for the error: Corrupted add-ons. Excel opening a file that was created in another application or obtaining data from an active source.28 Nov 2023

An OLE object in a computer

OLE (Object Linking and Embedding) is a component document technology from Microsoft that allows you to dynamically link files and applications together. An object is a combination of data and the application needed to modify that data.6 Oct 2021

Formula of OLE

Finally, OLE is calculated by multiplying the percentages of Availability, Performance, and Quality. Using the above examples: $OLE = 90\% \text{ (Availability)} * 90\% \text{ (Performance)} * 97.78\% \text{ (Quality)} = 79.1\%$.24 Feb 2024

How to use OLE?

To use OLE, you need both source and destination applications that support OLE. Both linking and embedding insert information from one document into another document. Also, both linked and embedded OLE objects can be edited from within the destination application.

What are the examples of OLE object?

Example 1 - a Word file:

DOC file. This file was then inserted as an OLE object. Note that in the output this is actually a graphic, not text! The object has been inserted in a single-cell table with a border and centered to distinguish it from the rest of the text.

Persistence

In object-oriented programming (OOP), "persistence" refers to the ability of an object to maintain its state (data) even after the program that created it has ended, essentially meaning the object's data can be stored and retrieved across multiple program sessions, typically by using a database to save the object's information to storage like a hard drive.

Key points about persistence in OOP:

□ Object lifetime beyond program execution:

Unlike temporary variables that only exist within a function call, persistent objects can

"survive" beyond the current program run, allowing data to be reused in future sessions.

□ Database interaction:

To achieve persistence, most OOP languages rely on database management systems (DBMS) to store and retrieve object data, usually through a process called "object-relational mapping" (ORM) which translates object properties to database tables and columns.

□ Implementation methods:

□ Serialization: Converting an object's state into a format that can be stored in a file or database (e.g., JSON, XML).

□ ORM frameworks: Libraries that simplify the process of mapping objects to database tables, handling data access operations automatically.

□ Challenges with persistence:

□ Impedance mismatch: Converting complex object relationships to a relational database structure can be challenging.

□ Performance considerations: Frequent database interactions can impact application speed.

Example of persistence in practice:

□ User profile data:

In a web application, a user's profile information (name, email, password) is often stored as persistent objects in a database so that the data is available when the user logs back in.

□ Shopping cart:

In an online store, a user's shopping cart contents are usually kept as persistent objects to maintain the items selected across multiple web pages.

Window Toolkit

In an object-oriented programming language, a "Window Toolkit" refers to a collection of classes that provide the building blocks for creating graphical user interfaces (GUIs), allowing developers to design windows with elements like buttons, text boxes, labels, and more, by instantiating and manipulating objects from these classes; a prominent example is Java's "Abstract Window Toolkit (AWT)" which offers a set of classes to build windows and their components on different operating systems with a consistent interface.

Key points about a Window Toolkit in OOP:

□ Object-Oriented Design:

Each visual element on a window (like a button, text field, or checkbox) is represented as an object with its own properties (size, color, text) and methods (like handling user clicks or text input).

□ Inheritance:

Classes representing different window elements often inherit from a common base class, allowing for shared functionality and consistent behavior across the UI.

□ Event Handling:

The toolkit provides mechanisms to capture user interactions (mouse clicks, keyboard presses) and trigger corresponding functions within your application logic through event listener objects.

□ Layout Managers:

To manage the arrangement of UI elements within a window, the toolkit often includes layout managers that automatically position components based on rules like grid or flow layouts.

Example concepts in a Window Toolkit:

□ Window Class: The primary class representing a visible window on the screen.

□ Panel Class: A container within a window to group related UI elements and manage their layout.

□ Button Class: Represents a clickable button with text or an icon.

□ Label Class: Displays static text on the window.

□ TextField Class: Allows users to enter text

□ Event Listener Interfaces: Interfaces that define methods to be called when specific events (like mouse clicks or keyboard presses) occur.

How to use a Window Toolkit:

1. Import necessary classes: Include the relevant classes from the toolkit library in your code.
2. Create window objects: Instantiate the desired window class and configure its properties like title and size.
3. Add UI components: Create objects of different UI elements (buttons, labels, text fields) and add them to the window using methods like `add()`.
4. Set layout: Use a layout manager to define how the elements should be arranged within the window.
5. Add event listeners: Register event listeners to handle user interactions with the UI elements.

Laboratory exercises in an Object-Oriented Programming language.

Laboratory exercises in an Object-Oriented Programming (OOP) language typically involve creating classes and objects to model real-world entities, utilizing key OOP concepts like encapsulation, inheritance, polymorphism, and abstraction through practical coding tasks, such as:

Basic OOP Concepts:

□ Creating a simple class:

Design a class representing a "Person" with attributes like name, age, and methods to introduce themselves or calculate their age in years.

□ Encapsulation:

Create a "BankAccount" class with private attributes for balance and methods to deposit, withdraw, and check balance, ensuring data protection through access modifiers.

Inheritance:

□ Employee hierarchy:

Design a base "Employee" class with common attributes, then create derived classes like "Manager" and "Developer" with specialized attributes and methods.

□ Geometric shapes:

Create a "Shape" base class with methods to calculate area and perimeter, then inherit to

create specific shapes like "Circle", "Rectangle", and "Triangle".

Polymorphism:

□ Animal behavior:

Create an "Animal" class with a "makeSound" method, then subclass to create different animals like "Dog", "Cat", and "Bird" with their own unique sounds.

□ Shape calculations:

Implement a "calculateArea" method in the base "Shape" class which can be overridden in derived classes to calculate area based on their specific properties.

Advanced Concepts:

□ Abstract classes:

Design an abstract "Shape" class with abstract methods for area and perimeter, forcing subclasses to implement these methods.

□ Interfaces:

Create an "IShape" interface defining methods for area and perimeter, which can be implemented by various concrete shape classes.

□ Collections and data structures:

Use OOP concepts to create classes representing data structures like "LinkedList", "Stack", or "Queue".

Real-world application scenarios:

□ Inventory management:

Create classes for products, orders, and customers to manage a virtual store inventory.

□ Point-of-sale system:

Design classes for items, transactions, and payment methods to simulate a retail checkout process.

□ Game development:

Implement classes for characters, items, and game environments to create a simple game simulation.

Important aspects of lab exercises:

□ Clear problem definition:

Each exercise should have a well-defined problem statement with specific requirements.

□ Testing and validation:

Write unit tests to verify the functionality of created classes and methods.

□ Code review and debugging:

Emphasize proper coding practices and debugging techniques to identify and fix errors in the code.

STRINGS

There are a handful of basic string manipulation tools that we use a lot when we work with text: Transform uppercase characters to lowercase (or vice versa). Replace a substring with another or delete the substring. Split a string into pieces at a particular character.

Character manipulation can refer to the behaviour of a character who controls others, or to functions that manipulate character strings.

Character manipulation as behaviour

- Controlling: The manipulator tries to be in charge and avoid responsibility
- Harmful: The manipulator uses tactics to cause confusion, doubt, or guilt
- Deceptive: The manipulator lies, blames, criticizes, or shames others
- Guilt-tripping: The manipulator makes others feel bad so they do what the manipulator wants
- Gaslighting: The manipulator convinces others that their reality is untrue
- Love-bombing: The manipulator showers others with affection to gain control
- Projection: The manipulator puts their own feelings onto others

Character manipulation as functions

- CONCAT: A character-manipulation function
- SUBSTR: A character-manipulation function
- INSTR: A character-manipulation function
- LPAD: A character-manipulation function
- RPAD: A character-manipulation function
- TRIM: A character-manipulation function
- REPLACE: A character-manipulation function

String Manipulation is a class of problems where a user is asked to process a given string and use/change its data. An example question would be a great way to understand the problems that are usually classified under this category.

What is string manipulation instruction?

A series of data byte or word available in memory at consecutive locations, to be referred as Byte String or Word String. A String of characters may be located in consecutive memory locations, where each character may be represented by its ASCII equivalent.

How does string manipulation work?

In simple terms, it refers to the manipulation of textual data to achieve desired results. At its core, string manipulation involves applying various operations to strings, such as concatenation, splitting, searching, replacing, and transforming.

An immutable object is an object that cannot be modified after it is created. This is different from a mutable object, which can be changed after it is created.

Examples of immutable objects

- String: In Java, the value of a String object never changes after it is instantiated.
- Integer: The content of an Integer object never changes.

Benefits of immutable objects

- Reliable code: Immutable objects are a good strategy for creating simple, reliable code.
- Thread safety: Immutable classes are inherently thread-safe because their state cannot be changed after creation.
- Caching: Immutable objects can be safely cached because their state doesn't change.
- Testable code: Code that uses immutable objects is simpler to test, which leads to a more robust and error free code base.

How immutable objects work

In functional programming, functions operate by taking an input and returning a new object, without modifying the original object.

OBJECT COLLECTION

In object-oriented programming, an "Object Collection" refers to a data structure that allows you to store and manage a group of objects, with "ArrayList" being a common implementation in languages like Java, providing efficient methods to add, access, and remove elements from a dynamic array; while "linked lists" use a chain of nodes, each containing data and a pointer to the next node, making insertions and deletions at any position more efficient than arrays, but slower for random access; and "multi-dimensional arrays" are arrays that hold arrays within them, allowing for data organization in multiple dimensions like rows and columns.

Key points about each concept:

□ ArrayList:-

□ Implementation: Uses a dynamic array to store elements, meaning the size can grow as needed.

□ Access: Allows fast random access to elements using their index.

□ Operations: Common operations include adding elements (add), retrieving elements by index (get), removing elements (remove), and checking size (size).

□ Best Use Case: When you need quick access to elements based on their position in the list.

□ Linked Lists:-

□ Structure: Each element (called a "node") contains data and a pointer to the next node in the sequence.

□ Node Class: A basic linked list is often implemented with a "Node" class which holds data and a reference to the next node.

□ Operations: Insertion and deletion at the beginning or middle of the list are efficient.

□ Best Use Case: When frequent insertions or deletions are needed at various positions within the list.

□ Multi-dimensional Arrays:-

□ Concept: An array that contains other arrays, creating a structure with multiple dimensions (like rows and columns in a matrix).

□ Access: Elements are accessed using multiple indices, one for each dimension.

□ Example: A 2D array can represent a table with rows and columns.

□ Best Use Case: When you need to store data in a grid-like format where each element can be accessed using multiple coordinates

Paper 4 of 6 — CSC 314 Object-Oriented Programming: (3 Units) Course Handout

Level 300 • First Semester • Handout

CSC 314 Object-Oriented Programming: (3 Units)

A MUSTAPHER

Review of CSC121 concepts: Procedural programming, iterative development of “growing” algorithms, parallel arrays. File input and output: Character-oriented file I/O, line-oriented file I/O. Basic OOP Concepts: Classes, Objects, inheritance, polymorphism, Data Abstraction, Tools for developing, Compiling, interpreting and debugging, Java Programs, Java Syntax and data objects, operators. Control flow constructs, objects and classes programming, Arrays, methods.

B STAR

Exceptions, Applets and the Abstract, OLE, Persistence, Window Toolkit, Laboratory exercises in an OOP Language. Strings: Character manipulation, string manipulation, immutable objects. Object Collections: The ArrayList class, ArrayList operations. Lists. Organization of linked lists, linked lists using a Node class. Multi-dimensional arrays.

EXCEPTIONS

In an Object-Oriented Programming (OOP) language, an "exception" is a mechanism used to signal an unexpected error or abnormal condition that occurs during program execution, allowing the program to gracefully handle the issue instead of crashing; it is represented as an object that can be caught and dealt with using specific code blocks, enabling centralized error handling and improved code readability compared to traditional error codes.

Key points about exceptions in OOP:

□ Exception object:

When an error occurs, the program throws an exception object, which contains information about the error type and details about the situation.

□ Try-Catch blocks:

Code that might potentially throw an exception is placed within a "try" block, and the code to handle the exception (if thrown) is placed within a corresponding "catch" block.

□ Exception hierarchy:

Most OOP languages have a hierarchy of exception classes, allowing for specific handling of different types of errors.

□ Custom exceptions:

Developers can create their own exception classes to represent application-specific errors, providing more context and clarity for error handling.

Common examples of exceptions in OOP languages:

- ZeroDivisionError: Attempting to divide by zero.
- IndexError: Accessing an index outside the bounds of a list or array.
- FileNotFoundError: Trying to open a file that doesn't exist.

□ `TypeError`: Trying to use a value of the wrong data type.

□ `NullPointerException`: Accessing a null reference as if it were a valid object.

Benefits of using exceptions in OOP:

□ Improved code readability:

Separates error handling logic from the main program flow, making code easier to understand.

□ Flexibility:

Allows for different error handling strategies depending on the type of exception thrown.

□ Error propagation:

Exceptions can be propagated up the call stack, allowing higher-level functions to handle errors that occur in lower-level functions.

APPLET

An applet is a small program that is intended not to be run on its own, but rather to be embedded inside another application. The `Applet` class must be the superclass of any applet that is to be embedded in a Web page or viewed by the Java Applet Viewer. An applet is a small computer program that performs a specific task. It is typically embedded within another larger app or software platform and has limited functionality. This allows applets to run quickly and reliably without demanding a lot of system resources.

What are the two types of applets?

In Java, there are two types of Applet

□ Java Applets based on the AWT(Abstract Window Toolkit) packages by extending its `Applet` class.

□ Java Applets is based on the Swing package by extending its `JApplet` Class in it.

Local applet

This is a type of applet specific to Java applets. Local applets are created and saved on the local system. When an HTML file contains a local applet, a web page does not need to get data from the internet. The filename or pathname within the code specifies or defines it.

What are the five main methods of applet?

There are five main methods in the applet life cycle. These are: `init()`, `start()`, `stop()`, `paint()` and `destroy()`. These methods are invoked for execution. An applet is also a class Java which is created by extending the class java.

Abstract class

Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class). Abstract method: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

What are the 4 basics of OOP abstraction?

OOP allows objects to interact with each other using four basic principles: encapsulation, inheritance, polymorphism, and abstraction. These four OOP principles enable objects to communicate and collaborate to create powerful applications

Example of an abstract data type in OOP

ADT is a container which holds different types of objects with specifications. logical representation(i.e an interface or protocol) of the data and the operations to manipulate the component elements of the data. Examples of ADT: List, Map, Set, Stack, Queue, Tree, Graph

The abstract keyword in Java is used to declare a class or a method that cannot be instantiated directly or must be implemented by subclasses, respectively. It is a key part of Java's abstraction mechanism, allowing developers to define abstract classes and methods that provide a blueprint for other classes.

An example of abstraction in OOPS is the use of abstract classes and methods. Abstract classes serve as templates or blueprints for objects but cannot be instantiated on their own. Abstract methods defined in these classes lack implementation details and must be provided by concrete (non-abstract) subclasses.

Abstraction is the progress of modelling real-world objects into programming language. Abstraction is not about interfaces or abstract classes.

Hence interfaces and abstract classes are just two techniques used in this progress.

In an Object-Oriented Programming language like Java, everything is an abstraction: interface, class, field, method, variable, etc.

Abstraction is the fundamental concept on which other concepts rely: encapsulation, inheritance, and polymorphism

Definition 2: Abstraction is one of the key concepts of object-oriented programming (OOP) languages. Its main goal is to handle complexity by hiding unnecessary details from the user. That enables the user to implement more complex logic on top of the provided abstraction without understanding or even thinking about all the hidden complexity.

OLE Object Linking and Embedding

In the context of Object Oriented Programming (OOP), "OLE" stands for "Object Linking and Embedding," which refers to a Microsoft technology that allows users to embed objects created in one application within another application, essentially letting you integrate data from different programs into a single document; for example, inserting an Excel spreadsheet into a Word document.

Object Linking and Embedding (OLE) is a proprietary technology developed by Microsoft that allows embedding and linking to documents and other objects.

Key points about OLE:

□ Functionality:

OLE enables users to "link" or "embed" objects, meaning the data can either be dynamically updated from the source application when changes are made, or be copied directly into the container application as a static element.

□ Implementation:

Within an OOP language, OLE functionality is typically achieved through specific libraries or APIs provided by the operating system, allowing developers to create applications that

can interact with objects from other applications.

□ Example scenario:

Imagine creating a presentation in PowerPoint where you want to embed a live graph from Excel. Using OLE, you could insert the Excel chart directly into the slide, and any changes made to the Excel source file would be reflected in the presentation.

OLE (Object Linking and Embedding) objects are active links in your project to documents created and edited in other applications. This enables you to insert formatted objects that you cannot create directly in the Help+Manual editor, such as formulas created in the MS Equation Editor.

OLE operation

Object Linking and Embedding (OLE) is a feature that allows Office applications to interact with other applications. Common reasons for the error: Corrupted add-ons. Excel opening a file that was created in another application or obtaining data from an active source. 28 Nov 2023

An OLE object in a computer

OLE (Object Linking and Embedding) is a component document technology from Microsoft that allows you to dynamically link files and applications together. An object is a combination of data and the application needed to modify that data. 6 Oct 2021

Formula of OLE

Finally, OLE is calculated by multiplying the percentages of Availability, Performance, and Quality. Using the above examples: $OLE = 90\% \text{ (Availability)} * 90\% \text{ (Performance)} * 97.78\% \text{ (Quality)} = 79.1\%$. 24 Feb 2024

How to use OLE?

To use OLE, you need both source and destination applications that support OLE. Both linking and embedding insert information from one document into another document. Also, both linked and embedded OLE objects can be edited from within the destination application.

What are the examples of OLE object?

Example 1 - a Word file:

DOC file. This file was then inserted as an OLE object. Note that in the output this is actually a graphic, not text! The object has been inserted in a single-cell table with a border and centered to distinguish it from the rest of the text.

Persistence

In object-oriented programming (OOP), "persistence" refers to the ability of an object to maintain its state (data) even after the program that created it has ended, essentially meaning the object's data can be stored and retrieved across multiple program sessions, typically by using a database to save the object's information to storage like a hard drive.

Key points about persistence in OOP:

□ Object lifetime beyond program execution:

Unlike temporary variables that only exist within a function call, persistent objects can

"survive" beyond the current program run, allowing data to be reused in future sessions.

□ Database interaction:

To achieve persistence, most OOP languages rely on database management systems (DBMS) to store and retrieve object data, usually through a process called "object-relational mapping" (ORM) which translates object properties to database tables and columns.

□ Implementation methods:

□ Serialization: Converting an object's state into a format that can be stored in a file or database (e.g., JSON, XML).

□ ORM frameworks: Libraries that simplify the process of mapping objects to database tables, handling data access operations automatically.

□ Challenges with persistence:

□ Impedance mismatch: Converting complex object relationships to a relational database structure can be challenging.

□ Performance considerations: Frequent database interactions can impact application speed.

Example of persistence in practice:

□ User profile data:

In a web application, a user's profile information (name, email, password) is often stored as persistent objects in a database so that the data is available when the user logs back in.

□ Shopping cart:

In an online store, a user's shopping cart contents are usually kept as persistent objects to maintain the items selected across multiple web pages.

Window Toolkit

In an object-oriented programming language, a "Window Toolkit" refers to a collection of classes that provide the building blocks for creating graphical user interfaces (GUIs), allowing developers to design windows with elements like buttons, text boxes, labels, and more, by instantiating and manipulating objects from these classes; a prominent example is Java's "Abstract Window Toolkit (AWT)" which offers a set of classes to build windows and their components on different operating systems with a consistent interface.

Key points about a Window Toolkit in OOP:

□ Object-Oriented Design:

Each visual element on a window (like a button, text field, or checkbox) is represented as an object with its own properties (size, color, text) and methods (like handling user clicks or text input).

□ Inheritance:

Classes representing different window elements often inherit from a common base class, allowing for shared functionality and consistent behavior across the UI.

□ Event Handling:

The toolkit provides mechanisms to capture user interactions (mouse clicks, keyboard presses) and trigger corresponding functions within your application logic through event listener objects.

□ Layout Managers:

To manage the arrangement of UI elements within a window, the toolkit often includes layout managers that automatically position components based on rules like grid or flow layouts.

Example concepts in a Window Toolkit:

□ Window Class: The primary class representing a visible window on the screen.

□ Panel Class: A container within a window to group related UI elements and manage their layout.

□ Button Class: Represents a clickable button with text or an icon.

□ Label Class: Displays static text on the window.

□ TextField Class: Allows users to enter text

□ Event Listener Interfaces: Interfaces that define methods to be called when specific events (like mouse clicks or keyboard presses) occur.

How to use a Window Toolkit:

1. Import necessary classes: Include the relevant classes from the toolkit library in your code.
2. Create window objects: Instantiate the desired window class and configure its properties like title and size.
3. Add UI components: Create objects of different UI elements (buttons, labels, text fields) and add them to the window using methods like `add()`.
4. Set layout: Use a layout manager to define how the elements should be arranged within the window.
5. Add event listeners: Register event listeners to handle user interactions with the UI elements.

Laboratory exercises in an Object-Oriented Programming language.

Laboratory exercises in an Object-Oriented Programming (OOP) language typically involve creating classes and objects to model real-world entities, utilizing key OOP concepts like encapsulation, inheritance, polymorphism, and abstraction through practical coding tasks, such as:

Basic OOP Concepts:

□ Creating a simple class:

Design a class representing a "Person" with attributes like name, age, and methods to introduce themselves or calculate their age in years.

□ Encapsulation:

Create a "BankAccount" class with private attributes for balance and methods to deposit, withdraw, and check balance, ensuring data protection through access modifiers.

Inheritance:

□ Employee hierarchy:

Design a base "Employee" class with common attributes, then create derived classes like "Manager" and "Developer" with specialized attributes and methods.

□ Geometric shapes:

Create a "Shape" base class with methods to calculate area and perimeter, then inherit to

create specific shapes like "Circle", "Rectangle", and "Triangle".

Polymorphism:

□ Animal behavior:

Create an "Animal" class with a "makeSound" method, then subclass to create different animals like "Dog", "Cat", and "Bird" with their own unique sounds.

□ Shape calculations:

Implement a "calculateArea" method in the base "Shape" class which can be overridden in derived classes to calculate area based on their specific properties.

Advanced Concepts:

□ Abstract classes:

Design an abstract "Shape" class with abstract methods for area and perimeter, forcing subclasses to implement these methods.

□ Interfaces:

Create an "IShape" interface defining methods for area and perimeter, which can be implemented by various concrete shape classes.

□ Collections and data structures:

Use OOP concepts to create classes representing data structures like "LinkedList", "Stack", or "Queue".

Real-world application scenarios:

□ Inventory management:

Create classes for products, orders, and customers to manage a virtual store inventory.

□ Point-of-sale system:

Design classes for items, transactions, and payment methods to simulate a retail checkout process.

□ Game development:

Implement classes for characters, items, and game environments to create a simple game simulation.

Important aspects of lab exercises:

□ Clear problem definition:

Each exercise should have a well-defined problem statement with specific requirements.

□ Testing and validation:

Write unit tests to verify the functionality of created classes and methods.

□ Code review and debugging:

Emphasize proper coding practices and debugging techniques to identify and fix errors in the code.

STRINGS

There are a handful of basic string manipulation tools that we use a lot when we work with text: Transform uppercase characters to lowercase (or vice versa). Replace a substring with another or delete the substring. Split a string into pieces at a particular character.

Character manipulation can refer to the behaviour of a character who controls others, or to functions that manipulate character strings.

Character manipulation as behaviour

- Controlling: The manipulator tries to be in charge and avoid responsibility
- Harmful: The manipulator uses tactics to cause confusion, doubt, or guilt
- Deceptive: The manipulator lies, blames, criticizes, or shames others
- Guilt-tripping: The manipulator makes others feel bad so they do what the manipulator wants
- Gaslighting: The manipulator convinces others that their reality is untrue
- Love-bombing: The manipulator showers others with affection to gain control
- Projection: The manipulator puts their own feelings onto others

Character manipulation as functions

- CONCAT: A character-manipulation function
- SUBSTR: A character-manipulation function
- INSTR: A character-manipulation function
- LPAD: A character-manipulation function
- RPAD: A character-manipulation function
- TRIM: A character-manipulation function
- REPLACE: A character-manipulation function

String Manipulation is a class of problems where a user is asked to process a given string and use/change its data. An example question would be a great way to understand the problems that are usually classified under this category.

What is string manipulation instruction?

A series of data byte or word available in memory at consecutive locations, to be referred as Byte String or Word String. A String of characters may be located in consecutive memory locations, where each character may be represented by its ASCII equivalent.

How does string manipulation work?

In simple terms, it refers to the manipulation of textual data to achieve desired results. At its core, string manipulation involves applying various operations to strings, such as concatenation, splitting, searching, replacing, and transforming.

An immutable object is an object that cannot be modified after it is created. This is different from a mutable object, which can be changed after it is created.

Examples of immutable objects

- String: In Java, the value of a String object never changes after it is instantiated.
- Integer: The content of an Integer object never changes.

Benefits of immutable objects

- Reliable code: Immutable objects are a good strategy for creating simple, reliable code.
- Thread safety: Immutable classes are inherently thread-safe because their state cannot be changed after creation.
- Caching: Immutable objects can be safely cached because their state doesn't change.
- Testable code: Code that uses immutable objects is simpler to test, which leads to a more robust and error free code base.

How immutable objects work

In functional programming, functions operate by taking an input and returning a new object, without modifying the original object.

OBJECT COLLECTION

In object-oriented programming, an "Object Collection" refers to a data structure that allows you to store and manage a group of objects, with "ArrayList" being a common implementation in languages like Java, providing efficient methods to add, access, and remove elements from a dynamic array; while "linked lists" use a chain of nodes, each containing data and a pointer to the next node, making insertions and deletions at any position more efficient than arrays, but slower for random access; and "multi-dimensional arrays" are arrays that hold arrays within them, allowing for data organization in multiple dimensions like rows and columns.

Key points about each concept:

□ ArrayList:-

□ Implementation: Uses a dynamic array to store elements, meaning the size can grow as needed.

□ Access: Allows fast random access to elements using their index.

□ Operations: Common operations include adding elements (add), retrieving elements by index (get), removing elements (remove), and checking size (size).

□ Best Use Case: When you need quick access to elements based on their position in the list.

□ Linked Lists:-

□ Structure: Each element (called a "node") contains data and a pointer to the next node in the sequence.

□ Node Class: A basic linked list is often implemented with a "Node" class which holds data and a reference to the next node.

□ Operations: Insertion and deletion at the beginning or middle of the list are efficient.

□ Best Use Case: When frequent insertions or deletions are needed at various positions within the list.

□ Multi-dimensional Arrays:-

□ Concept: An array that contains other arrays, creating a structure with multiple dimensions (like rows and columns in a matrix).

□ Access: Elements are accessed using multiple indices, one for each dimension.

□ Example: A 2D array can represent a table with rows and columns.

□ Best Use Case: When you need to store data in a grid-like format where each element can be accessed using multiple coordinates

Paper 5 of 6 — CSC 314 Object-Oriented Programming: (3 Units)

Level 300 • First Semester • Handout

CSC 314 Object-Oriented Programming: (3 Units)

A MUSTAPHER

Review of CSC121 concepts: Procedural programming, iterative development of “growing” algorithms, parallel arrays. File input and output: Character-oriented file I/O, line-oriented file I/O. Basic OOP Concepts: Classes, Objects, inheritance, polymorphism, Data Abstraction, Tools for developing, Compiling, interpreting and debugging, Java Programs, Java Syntax and data objects, operators. Control flow constructs, objects and classes programming, Arrays, methods.

B STAR

Exceptions, Applets and the Abstract, OLE, Persistence, Window Toolkit, Laboratory exercises in an OOP Language. Strings: Character manipulation, string manipulation, immutable objects.

Object Collections: The ArrayList class, ArrayList operations. Lists. Organization of linked lists, linked lists using a Node class. Multi-dimensional arrays.

EXCEPTIONS

In an Object-Oriented Programming (OOP) language, an "exception" is a mechanism used to signal an unexpected error or abnormal condition that occurs during program execution, allowing the program to gracefully handle the issue instead of crashing; it is represented as an object that can be caught and dealt with using specific code blocks, enabling centralized error handling and improved code readability compared to traditional error codes.

Key points about exceptions in OOP:

- Exception object:

When an error occurs, the program throws an exception object, which contains information about the error type and details about the situation.

- Try-Catch blocks:

Code that might potentially throw an exception is placed within a "try" block, and the code to handle the exception (if thrown) is placed within a corresponding "catch" block.

- Exception hierarchy:

Most OOP languages have a hierarchy of exception classes, allowing for specific handling of different types of errors.

- Custom exceptions:

Developers can create their own exception classes to represent application-specific errors, providing more context and clarity for error handling.

Common examples of exceptions in OOP languages:

- ZeroDivisionError: Attempting to divide by zero.

- **IndexError:** Accessing an index outside the bounds of a list or array.
- **FileNotFoundError:** Trying to open a file that doesn't exist.
- **TypeError:** Trying to use a value of the wrong data type.
- **NullPointerException:** Accessing a null reference as if it were a valid object.

Benefits of using exceptions in OOP:

- **Improved code readability:**

Separates error handling logic from the main program flow, making code easier to understand.

- **Flexibility:**

Allows for different error handling strategies depending on the type of exception thrown.

- **Error propagation:**

Exceptions can be propagated up the call stack, allowing higher-level functions to handle errors that occur in lower-level functions.

APPLET

An applet is a small program that is intended not to be run on its own, but rather to be embedded inside another application. The Applet class must be the superclass of any applet that is to be embedded in a Web page or viewed by the Java Applet Viewer. An applet is a small computer program that performs a specific task. It is typically embedded within another larger app or software platform and has limited functionality. This allows applets to run quickly and reliably without demanding a lot of system resources.

What are the two types of applets?

In Java, there are two types of Applet

- Java Applets based on the AWT(Abstract Window Toolkit) packages by extending its Applet class.
- Java Applets is based on the Swing package by extending its JApplet Class in it.

Local applet

This is a type of applet specific to Java applets. Local applets are created and saved on the local system. When an HTML file contains a local applet, a web page does not need to get data from the internet. The filename or pathname within the code specifies or defines it.

What are the five main methods of applet?

There are five main methods in the applet life cycle. These are: `init()`, `start()`, `stop()`, `paint()` and `destroy()`. These methods are invoked for execution. An applet is also a class Java which is created by extending the class java.

Abstract class

Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class). **Abstract method:** can only be used in an

abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

What are the 4 basics of OOP abstraction?

OOP allows objects to interact with each other using four basic principles: encapsulation, inheritance, polymorphism, and abstraction. These four OOP principles enable objects to communicate and collaborate to create powerful applications

Example of an abstract data type in OOP

ADT is a container which holds different types of objects with specifications. logical representation(i.e an interface or protocol) of the data and the operations to manipulate the component elements of the data. Examples of ADT: List, Map, Set, Stack, Queue, Tree, Graph

The abstract keyword in Java is used to declare a class or a method that cannot be instantiated directly or must be implemented by subclasses, respectively. It is a key part of Java's abstraction mechanism, allowing developers to define abstract classes and methods that provide a blueprint for other classes.

An example of abstraction in OOPS is the use of abstract classes and methods. Abstract classes serve as templates or blueprints for objects but cannot be instantiated on their own. Abstract methods defined in these classes lack implementation details and must be provided by concrete (non-abstract) subclasses.

Abstraction is the progress of modeling real-world objects into programming language. Abstraction is not about interfaces or abstract classes.

Hence interfaces and abstract classes are just two techniques used in this progress. In an Object-Oriented Programming language like Java, everything is an abstraction: interface, class, field, method, variable, etc.

Abstraction is the fundamental concept on which other concepts rely: encapsulation, inheritance, and polymorphism

Definition 2: Abstraction is one of the key concepts of object-oriented programming (OOP) languages. Its main goal is to handle complexity by hiding unnecessary details from the user. That enables the user to implement more complex logic on top of the provided abstraction without understanding or even thinking about all the hidden complexity.

OLE Object Linking and Embedding

In the context of Object Oriented Programming (OOP), "OLE" stands for "Object Linking and Embedding," which refers to a Microsoft technology that allows users to embed objects created in one application within another application, essentially letting you integrate data from different programs into a single document; for example, inserting an Excel spreadsheet into a Word document.

Object Linking and Embedding (OLE) is a proprietary technology developed by Microsoft

that allows embedding and linking to documents and other objects.

Key points about OLE:

- **Functionality:**

OLE enables users to "link" or "embed" objects, meaning the data can either be dynamically updated from the source application when changes are made, or be copied directly into the container application as a static element.

- **Implementation:**

Within an OOP language, OLE functionality is typically achieved through specific libraries or APIs provided by the operating system, allowing developers to create applications that can interact with objects from other applications.

- **Example scenario:**

Imagine creating a presentation in PowerPoint where you want to embed a live graph from Excel. Using OLE, you could insert the Excel chart directly into the slide, and any changes made to the Excel source file would be reflected in the presentation.

OLE (Object Linking and Embedding) objects are active links in your project to documents created and edited in other applications. This enables you to insert formatted objects that you cannot create directly in the Help+Manual editor, such as formulas created in the MS Equation Editor.

OLE operation

Object Linking and Embedding (OLE) is a feature that allows Office applications to interact with other applications. Common reasons for the error: Corrupted add-ons. Excel opening a file that was created in another application or obtaining data from an active source. 28 Nov 2023

An OLE object in a computer

OLE (Object Linking and Embedding) is a component document technology from Microsoft that allows you to dynamically link files and applications together. An object is a combination of data and the application needed to modify that data. 6 Oct 2021

Formula of OLE

Finally, OLE is calculated by multiplying the percentages of Availability, Performance, and Quality. Using the above examples: $OLE = 90\% \text{ (Availability)} * 90\% \text{ (Performance)} * 97.78\% \text{ (Quality)} = 79.1\%$. 24 Feb 2024

How to use OLE?

To use OLE, you need both source and destination applications that support OLE. Both linking and embedding insert information from one document into another document. Also, both linked and embedded OLE objects can be edited from within the destination application.

What are the examples of OLE object?

Example 1 - a Word file:

DOC file. This file was then inserted as an OLE object. Note that in the output this is actually a graphic, not text! The object has been inserted in a single-cell table with a border and centered to distinguish it from the rest of the text.

Persistence

In object-oriented programming (OOP), "persistence" refers to the ability of an object to maintain its state (data) even after the program that created it has ended, essentially meaning the object's data can be stored and retrieved across multiple program sessions, typically by using a database to save the object's information to storage like a hard drive.

Key points about persistence in OOP:

- Object lifetime beyond program execution:

Unlike temporary variables that only exist within a function call, persistent objects can "survive" beyond the current program run, allowing data to be reused in future sessions.

- Database interaction:

To achieve persistence, most OOP languages rely on database management systems (DBMS) to store and retrieve object data, usually through a process called "object-relational mapping" (ORM) which translates object properties to database tables and columns.

- Implementation methods:

- Serialization: Converting an object's state into a format that can be stored in a file or database (e.g., JSON, XML).

- ORM frameworks: Libraries that simplify the process of mapping objects to database tables, handling data access operations automatically.

- Challenges with persistence:

- Impedance mismatch: Converting complex object relationships to a relational database structure can be challenging.

- Performance considerations: Frequent database interactions can impact application speed.

Example of persistence in practice:

- User profile data:

In a web application, a user's profile information (name, email, password) is often stored as persistent objects in a database so that the data is available when the user logs back in.

- Shopping cart:

In an online store, a user's shopping cart contents are usually kept as persistent objects to maintain the items selected across multiple web pages.

Window Toolkit

In an object-oriented programming language, a "Window Toolkit" refers to a collection of classes that provide the building blocks for creating graphical user interfaces (GUIs), allowing developers to design windows with elements like buttons, text boxes, labels, and more, by instantiating and manipulating objects from these classes; a prominent example is Java's "Abstract Window Toolkit (AWT)" which offers a set of classes to build windows and their components on different operating systems with a consistent interface.

Key points about a Window Toolkit in OOP:

- **Object-Oriented Design:**

Each visual element on a window (like a button, text field, or checkbox) is represented as an object with its own properties (size, color, text) and methods (like handling user clicks or text input).

- **Inheritance:**

Classes representing different window elements often inherit from a common base class, allowing for shared functionality and consistent behavior across the UI.

- **Event Handling:**

The toolkit provides mechanisms to capture user interactions (mouse clicks, keyboard presses) and trigger corresponding functions within your application logic through event listener objects.

- **Layout Managers:**

To manage the arrangement of UI elements within a window, the toolkit often includes layout managers that automatically position components based on rules like grid or flow layouts.

Example concepts in a Window Toolkit:

- **Window Class:** The primary class representing a visible window on the screen.
- **Panel Class:** A container within a window to group related UI elements and manage their layout.
- **Button Class:** Represents a clickable button with text or an icon.
- **Label Class:** Displays static text on the window.
- **TextField Class:** Allows users to enter text
- **Event Listener Interfaces:** Interfaces that define methods to be called when specific events (like mouse clicks or keyboard presses) occur.

How to use a Window Toolkit:

1. **Import necessary classes:** Include the relevant classes from the toolkit library in your code.
2. **Create window objects:** Instantiate the desired window class and configure its properties like title and size.

3. Add UI components: Create objects of different UI elements (buttons, labels, text fields) and add them to the window using methods like add().
4. Set layout: Use a layout manager to define how the elements should be arranged within the window.
5. Add event listeners: Register event listeners to handle user interactions with the UI elements.

Laboratory exercises in an Object-Oriented Programming language.

Laboratory exercises in an Object-Oriented Programming (OOP) language typically involve creating classes and objects to model real-world entities, utilizing key OOP concepts like encapsulation, inheritance, polymorphism, and abstraction through practical coding tasks, such as:

Basic OOP Concepts:

- Creating a simple class:

Design a class representing a "Person" with attributes like name, age, and methods to introduce themselves or calculate their age in years.

- Encapsulation:

Create a "BankAccount" class with private attributes for balance and methods to deposit, withdraw, and check balance, ensuring data protection through access modifiers.

Inheritance:

- Employee hierarchy:

Design a base "Employee" class with common attributes, then create derived classes like "Manager" and "Developer" with specialized attributes and methods.

- Geometric shapes:

Create a "Shape" base class with methods to calculate area and perimeter, then inherit to create specific shapes like "Circle", "Rectangle", and "Triangle".

Polymorphism:

- Animal behavior:

Create an "Animal" class with a "makeSound" method, then subclass to create different animals like "Dog", "Cat", and "Bird" with their own unique sounds.

- Shape calculations:

Implement a "calculateArea" method in the base "Shape" class which can be overridden in derived classes to calculate area based on their specific properties.

Advanced Concepts:

- Abstract classes:

Design an abstract "Shape" class with abstract methods for area and perimeter, forcing subclasses to implement these methods.

- Interfaces:

Create an "IShape" interface defining methods for area and perimeter, which can be implemented by various concrete shape classes.

- Collections and data structures:

Use OOP concepts to create classes representing data structures like "LinkedList", "Stack", or "Queue".

Real-world application scenarios:

- Inventory management:

Create classes for products, orders, and customers to manage a virtual store inventory.

- Point-of-sale system:

Design classes for items, transactions, and payment methods to simulate a retail checkout process.

- Game development:

Implement classes for characters, items, and game environments to create a simple game simulation.

Important aspects of lab exercises:

- Clear problem definition:

Each exercise should have a well-defined problem statement with specific requirements.

- Testing and validation:

Write unit tests to verify the functionality of created classes and methods.

- Code review and debugging:

Emphasize proper coding practices and debugging techniques to identify and fix errors in the code.

STRINGS

There are a handful of basic string manipulation tools that we use a lot when we work with text: Transform uppercase characters to lowercase (or vice versa). Replace a substring with another or delete the substring. Split a string into pieces at a particular character.

Character manipulation can refer to the behaviour of a character who controls others, or to functions that manipulate character strings.

Character manipulation as behaviour

- Controlling: The manipulator tries to be in charge and avoid responsibility
- Harmful: The manipulator uses tactics to cause confusion, doubt, or guilt
- Deceptive: The manipulator lies, blames, criticizes, or shames others
- Guilt-tripping: The manipulator makes others feel bad so they do what the manipulator wants
- Gaslighting: The manipulator convinces others that their reality is untrue
- Love-bombing: The manipulator showers others with affection to gain

control

- Projection: The manipulator puts their own feelings onto others

Character manipulation as functions

- CONCAT: A character-manipulation function
- SUBSTR: A character-manipulation function
- INSTR: A character-manipulation function
- LPAD: A character-manipulation function
- RPAD: A character-manipulation function
- TRIM: A character-manipulation function
- REPLACE: A character-manipulation function

String Manipulation is a class of problems where a user is asked to process a given string and use/change its data. An example question would be a great way to understand the problems that are usually classified under this category.

What is string manipulation instruction?

A series of data byte or word available in memory at consecutive locations, to be referred as Byte String or Word String. A String of characters may be located in consecutive memory locations, where each character may be represented by its ASCII equivalent.

How does string manipulation work?

In simple terms, it refers to the manipulation of textual data to achieve desired results. At its core, string manipulation involves applying various operations to strings, such as concatenation, splitting, searching, replacing, and transforming.

An immutable object is an object that cannot be modified after it is created. This is different from a mutable object, which can be changed after it is created.

Examples of immutable objects

- String: In Java, the value of a String object never changes after it is instantiated.
- Integer: The content of an Integer object never changes.

Benefits of immutable objects

- Reliable code: Immutable objects are a good strategy for creating simple, reliable code.
- Thread safety: Immutable classes are inherently thread-safe because their state cannot be changed after creation.
- Caching: Immutable objects can be safely cached because their state doesn't change.
- Testable code: Code that uses immutable objects is simpler to test, which leads to a more robust and error free code base.

How immutable objects work

In functional programming, functions operate by taking an input and

returning a new object, without modifying the original object.

OBJECT COLLECTION

In object-oriented programming, an "Object Collection" refers to a data structure that allows you to store and manage a group of objects, with "ArrayList" being a common implementation in languages like Java, providing efficient methods to add, access, and remove elements from a dynamic array; while "linked lists" use a chain of nodes, each containing data and a pointer to the next node, making insertions and deletions at any position more efficient than arrays, but slower for random access; and "multi-dimensional arrays" are arrays that hold arrays within them, allowing for data organization in multiple dimensions like rows and columns.

Key points about each concept:

- **ArrayList:-**
 - **Implementation:** Uses a dynamic array to store elements, meaning the size can grow as needed.
 - **Access:** Allows fast random access to elements using their index.
 - **Operations:** Common operations include adding elements (add), retrieving elements by index (get), removing elements (remove), and checking size (size).
 - **Best Use Case:** When you need quick access to elements based on their position in the list.
- **Linked Lists:-**
 - **Structure:** Each element (called a "node") contains data and a pointer to the next node in the sequence.
 - **Node Class:** A basic linked list is often implemented with a "Node" class which holds data and a reference to the next node.
 - **Operations:** Insertion and deletion at the beginning or middle of the list are efficient.
 - **Best Use Case:** When frequent insertions or deletions are needed at various positions within the list.
- **Multi-dimensional Arrays:-**
 - **Concept:** An array that contains other arrays, creating a structure with multiple dimensions (like rows and columns in a matrix).
 - **Access:** Elements are accessed using multiple indices, one for each dimension.
 - **Example:** A 2D array can represent a table with rows and columns.
 - **Best Use Case:** When you need to store data in a grid-like format where each element can be accessed using multiple coordinates

Paper 6 of 6 — CSC 314 Object-Oriented Programming: (3 Units)

Level 300 • First Semester • Handout

CSC 314 Object-Oriented Programming: (3 Units)

A MUSTAPHER

Review of CSC121 concepts: Procedural programming, iterative development of “growing” algorithms, parallel arrays. File input and output: Character-oriented file I/O, line-oriented file I/O. Basic OOP Concepts: Classes, Objects, inheritance, polymorphism, Data Abstraction, Tools for developing, Compiling, interpreting and debugging, Java Programs, Java Syntax and data objects, operators. Control flow constructs, objects and classes programming, Arrays, methods.

B STAR

Exceptions, Applets and the Abstract, OLE, Persistence, Window Toolkit, Laboratory exercises in an OOP Language. Strings: Character manipulation, string manipulation, immutable objects.

Object Collections: The ArrayList class, ArrayList operations. Lists. Organization of linked lists, linked lists using a Node class. Multi-dimensional arrays.

EXCEPTIONS

In an Object-Oriented Programming (OOP) language, an "exception" is a mechanism used to signal an unexpected error or abnormal condition that occurs during program execution, allowing the program to gracefully handle the issue instead of crashing; it is represented as an object that can be caught and dealt with using specific code blocks, enabling centralized error handling and improved code readability compared to traditional error codes.

Key points about exceptions in OOP:

- Exception object:

When an error occurs, the program throws an exception object, which contains information about the error type and details about the situation.

- Try-Catch blocks:

Code that might potentially throw an exception is placed within a "try" block, and the code to handle the exception (if thrown) is placed within a corresponding "catch" block.

- Exception hierarchy:

Most OOP languages have a hierarchy of exception classes, allowing for specific handling of different types of errors.

- Custom exceptions:

Developers can create their own exception classes to represent application-specific errors, providing more context and clarity for error handling.

Common examples of exceptions in OOP languages:

- ZeroDivisionError: Attempting to divide by zero.

- **IndexError:** Accessing an index outside the bounds of a list or array.
- **FileNotFoundException:** Trying to open a file that doesn't exist.
- **TypeError:** Trying to use a value of the wrong data type.
- **NullPointerException:** Accessing a null reference as if it were a valid object.

Benefits of using exceptions in OOP:

- **Improved code readability:**

Separates error handling logic from the main program flow, making code easier to understand.

- **Flexibility:**

Allows for different error handling strategies depending on the type of exception thrown.

- **Error propagation:**

Exceptions can be propagated up the call stack, allowing higher-level functions to handle errors that occur in lower-level functions.

APPLET

An applet is a small program that is intended not to be run on its own, but rather to be embedded inside another application. The Applet class must be the superclass of any applet that is to be embedded in a Web page or viewed by the Java Applet Viewer. An applet is a small computer program that performs a specific task. It is typically embedded within another larger app or software platform and has limited functionality. This allows applets to run quickly and reliably without demanding a lot of system resources.

What are the two types of applets?

In Java, there are two types of Applet

- Java Applets based on the AWT(Abstract Window Toolkit) packages by extending its Applet class.
- Java Applets is based on the Swing package by extending its JApplet Class in it.

Local applet

This is a type of applet specific to Java applets. Local applets are created and saved on the local system. When an HTML file contains a local applet, a web page does not need to get data from the internet. The filename or pathname within the code specifies or defines it.

What are the five main methods of applet?

There are five main methods in the applet life cycle. These are: `init()`, `start()`, `stop()`, `paint()` and `destroy()`. These methods are invoked for execution. An applet is also a class Java which is created by extending the class java.

Abstract class

Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class). **Abstract method:** can only be used in an

abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

What are the 4 basics of OOP abstraction?

OOP allows objects to interact with each other using four basic principles: encapsulation, inheritance, polymorphism, and abstraction. These four OOP principles enable objects to communicate and collaborate to create powerful applications

Example of an abstract data type in OOP

ADT is a container which holds different types of objects with specifications. logical representation(i.e an interface or protocol) of the data and the operations to manipulate the component elements of the data. Examples of ADT: List, Map, Set, Stack, Queue, Tree, Graph

The abstract keyword in Java is used to declare a class or a method that cannot be instantiated directly or must be implemented by subclasses, respectively. It is a key part of Java's abstraction mechanism, allowing developers to define abstract classes and methods that provide a blueprint for other classes.

An example of abstraction in OOPS is the use of abstract classes and methods. Abstract classes serve as templates or blueprints for objects but cannot be instantiated on their own. Abstract methods defined in these classes lack implementation details and must be provided by concrete (non-abstract) subclasses.

Abstraction is the progress of modeling real-world objects into programming language. Abstraction is not about interfaces or abstract classes.

Hence interfaces and abstract classes are just two techniques used in this progress. In an Object-Oriented Programming language like Java, everything is an abstraction: interface, class, field, method, variable, etc.

Abstraction is the fundamental concept on which other concepts rely: encapsulation, inheritance, and polymorphism

Definition 2: Abstraction is one of the key concepts of object-oriented programming (OOP) languages. Its main goal is to handle complexity by hiding unnecessary details from the user. That enables the user to implement more complex logic on top of the provided abstraction without understanding or even thinking about all the hidden complexity.

OLE Object Linking and Embedding

In the context of Object Oriented Programming (OOP), "OLE" stands for "Object Linking and Embedding," which refers to a Microsoft technology that allows users to embed objects created in one application within another application, essentially letting you integrate data from different programs into a single document; for example, inserting an Excel spreadsheet into a Word document.

Object Linking and Embedding (OLE) is a proprietary technology developed by Microsoft

that allows embedding and linking to documents and other objects.

Key points about OLE:

- **Functionality:**

OLE enables users to "link" or "embed" objects, meaning the data can either be dynamically updated from the source application when changes are made, or be copied directly into the container application as a static element.

- **Implementation:**

Within an OOP language, OLE functionality is typically achieved through specific libraries or APIs provided by the operating system, allowing developers to create applications that can interact with objects from other applications.

- **Example scenario:**

Imagine creating a presentation in PowerPoint where you want to embed a live graph from Excel. Using OLE, you could insert the Excel chart directly into the slide, and any changes made to the Excel source file would be reflected in the presentation.

OLE (Object Linking and Embedding) objects are active links in your project to documents created and edited in other applications. This enables you to insert formatted objects that you cannot create directly in the Help+Manual editor, such as formulas created in the MS Equation Editor.

OLE operation

Object Linking and Embedding (OLE) is a feature that allows Office applications to interact with other applications. Common reasons for the error: Corrupted add-ons. Excel opening a file that was created in another application or obtaining data from an active source. 28 Nov 2023

An OLE object in a computer

OLE (Object Linking and Embedding) is a component document technology from Microsoft that allows you to dynamically link files and applications together. An object is a combination of data and the application needed to modify that data. 6 Oct 2021

Formula of OLE

Finally, OLE is calculated by multiplying the percentages of Availability, Performance, and Quality. Using the above examples: $OLE = 90\% \text{ (Availability)} * 90\% \text{ (Performance)} * 97.78\% \text{ (Quality)} = 79.1\%$. 24 Feb 2024

How to use OLE?

To use OLE, you need both source and destination applications that support OLE. Both linking and embedding insert information from one document into another document. Also, both linked and embedded OLE objects can be edited from within the destination application.

What are the examples of OLE object?

Example 1 - a Word file:

DOC file. This file was then inserted as an OLE object. Note that in the output this is actually a graphic, not text! The object has been inserted in a single-cell table with a border and centered to distinguish it from the rest of the text.

Persistence

In object-oriented programming (OOP), "persistence" refers to the ability of an object to maintain its state (data) even after the program that created it has ended, essentially meaning the object's data can be stored and retrieved across multiple program sessions, typically by using a database to save the object's information to storage like a hard drive.

Key points about persistence in OOP:

- Object lifetime beyond program execution:

Unlike temporary variables that only exist within a function call, persistent objects can "survive" beyond the current program run, allowing data to be reused in future sessions.

- Database interaction:

To achieve persistence, most OOP languages rely on database management systems (DBMS) to store and retrieve object data, usually through a process called "object-relational mapping" (ORM) which translates object properties to database tables and columns.

- Implementation methods:

- Serialization: Converting an object's state into a format that can be stored in a file or database (e.g., JSON, XML).

- ORM frameworks: Libraries that simplify the process of mapping objects to database tables, handling data access operations automatically.

- Challenges with persistence:

- Impedance mismatch: Converting complex object relationships to a relational database structure can be challenging.

- Performance considerations: Frequent database interactions can impact application speed.

Example of persistence in practice:

- User profile data:

In a web application, a user's profile information (name, email, password) is often stored as persistent objects in a database so that the data is available when the user logs back in.

- Shopping cart:

In an online store, a user's shopping cart contents are usually kept as persistent objects to maintain the items selected across multiple web pages.

Window Toolkit

In an object-oriented programming language, a "Window Toolkit" refers to a collection of classes that provide the building blocks for creating graphical user interfaces (GUIs), allowing developers to design windows with elements like buttons, text boxes, labels, and more, by instantiating and manipulating objects from these classes; a prominent example is Java's "Abstract Window Toolkit (AWT)" which offers a set of classes to build windows and their components on different operating systems with a consistent interface.

Key points about a Window Toolkit in OOP:

- **Object-Oriented Design:**

Each visual element on a window (like a button, text field, or checkbox) is represented as an object with its own properties (size, color, text) and methods (like handling user clicks or text input).

- **Inheritance:**

Classes representing different window elements often inherit from a common base class, allowing for shared functionality and consistent behavior across the UI.

- **Event Handling:**

The toolkit provides mechanisms to capture user interactions (mouse clicks, keyboard presses) and trigger corresponding functions within your application logic through event listener objects.

- **Layout Managers:**

To manage the arrangement of UI elements within a window, the toolkit often includes layout managers that automatically position components based on rules like grid or flow layouts.

Example concepts in a Window Toolkit:

- **Window Class:** The primary class representing a visible window on the screen.
- **Panel Class:** A container within a window to group related UI elements and manage their layout.
- **Button Class:** Represents a clickable button with text or an icon.
- **Label Class:** Displays static text on the window.
- **TextField Class:** Allows users to enter text
- **Event Listener Interfaces:** Interfaces that define methods to be called when specific events (like mouse clicks or keyboard presses) occur.

How to use a Window Toolkit:

1. **Import necessary classes:** Include the relevant classes from the toolkit library in your code.
2. **Create window objects:** Instantiate the desired window class and configure its properties like title and size.

3. Add UI components: Create objects of different UI elements (buttons, labels, text fields) and add them to the window using methods like add().
4. Set layout: Use a layout manager to define how the elements should be arranged within the window.
5. Add event listeners: Register event listeners to handle user interactions with the UI elements.

Laboratory exercises in an Object-Oriented Programming language.

Laboratory exercises in an Object-Oriented Programming (OOP) language typically involve creating classes and objects to model real-world entities, utilizing key OOP concepts like encapsulation, inheritance, polymorphism, and abstraction through practical coding tasks, such as:

Basic OOP Concepts:

- Creating a simple class:

Design a class representing a "Person" with attributes like name, age, and methods to introduce themselves or calculate their age in years.

- Encapsulation:

Create a "BankAccount" class with private attributes for balance and methods to deposit, withdraw, and check balance, ensuring data protection through access modifiers.

Inheritance:

- Employee hierarchy:

Design a base "Employee" class with common attributes, then create derived classes like "Manager" and "Developer" with specialized attributes and methods.

- Geometric shapes:

Create a "Shape" base class with methods to calculate area and perimeter, then inherit to create specific shapes like "Circle", "Rectangle", and "Triangle".

Polymorphism:

- Animal behavior:

Create an "Animal" class with a "makeSound" method, then subclass to create different animals like "Dog", "Cat", and "Bird" with their own unique sounds.

- Shape calculations:

Implement a "calculateArea" method in the base "Shape" class which can be overridden in derived classes to calculate area based on their specific properties.

Advanced Concepts:

- Abstract classes:

Design an abstract "Shape" class with abstract methods for area and perimeter, forcing subclasses to implement these methods.

- Interfaces:

Create an "IShape" interface defining methods for area and perimeter, which can be implemented by various concrete shape classes.

- Collections and data structures:

Use OOP concepts to create classes representing data structures like "LinkedList", "Stack", or "Queue".

Real-world application scenarios:

- Inventory management:

Create classes for products, orders, and customers to manage a virtual store inventory.

- Point-of-sale system:

Design classes for items, transactions, and payment methods to simulate a retail checkout process.

- Game development:

Implement classes for characters, items, and game environments to create a simple game simulation.

Important aspects of lab exercises:

- Clear problem definition:

Each exercise should have a well-defined problem statement with specific requirements.

- Testing and validation:

Write unit tests to verify the functionality of created classes and methods.

- Code review and debugging:

Emphasize proper coding practices and debugging techniques to identify and fix errors in the code.

STRINGS

There are a handful of basic string manipulation tools that we use a lot when we work with text: Transform uppercase characters to lowercase (or vice versa). Replace a substring with another or delete the substring. Split a string into pieces at a particular character.

Character manipulation can refer to the behaviour of a character who controls others, or to functions that manipulate character strings.

Character manipulation as behaviour

- Controlling: The manipulator tries to be in charge and avoid responsibility
- Harmful: The manipulator uses tactics to cause confusion, doubt, or guilt
- Deceptive: The manipulator lies, blames, criticizes, or shames others
- Guilt-tripping: The manipulator makes others feel bad so they do what the manipulator wants
- Gaslighting: The manipulator convinces others that their reality is untrue
- Love-bombing: The manipulator showers others with affection to gain

control

- Projection: The manipulator puts their own feelings onto others

Character manipulation as functions

- CONCAT: A character-manipulation function
- SUBSTR: A character-manipulation function
- INSTR: A character-manipulation function
- LPAD: A character-manipulation function
- RPAD: A character-manipulation function
- TRIM: A character-manipulation function
- REPLACE: A character-manipulation function

String Manipulation is a class of problems where a user is asked to process a given string and use/change its data. An example question would be a great way to understand the problems that are usually classified under this category.

What is string manipulation instruction?

A series of data byte or word available in memory at consecutive locations, to be referred as Byte String or Word String. A String of characters may be located in consecutive memory locations, where each character may be represented by its ASCII equivalent.

How does string manipulation work?

In simple terms, it refers to the manipulation of textual data to achieve desired results. At its core, string manipulation involves applying various operations to strings, such as concatenation, splitting, searching, replacing, and transforming.

An immutable object is an object that cannot be modified after it is created. This is different from a mutable object, which can be changed after it is created.

Examples of immutable objects

- String: In Java, the value of a String object never changes after it is instantiated.
- Integer: The content of an Integer object never changes.

Benefits of immutable objects

- Reliable code: Immutable objects are a good strategy for creating simple, reliable code.
- Thread safety: Immutable classes are inherently thread-safe because their state cannot be changed after creation.
- Caching: Immutable objects can be safely cached because their state doesn't change.
- Testable code: Code that uses immutable objects is simpler to test, which leads to a more robust and error free code base.

How immutable objects work

In functional programming, functions operate by taking an input and

returning a new object, without modifying the original object.

OBJECT COLLECTION

In object-oriented programming, an "Object Collection" refers to a data structure that allows you to store and manage a group of objects, with "ArrayList" being a common implementation in languages like Java, providing efficient methods to add, access, and remove elements from a dynamic array; while "linked lists" use a chain of nodes, each containing data and a pointer to the next node, making insertions and deletions at any position more efficient than arrays, but slower for random access; and "multi-dimensional arrays" are arrays that hold arrays within them, allowing for data organization in multiple dimensions like rows and columns.

Key points about each concept:

- **ArrayList:-**
 - **Implementation:** Uses a dynamic array to store elements, meaning the size can grow as needed.
 - **Access:** Allows fast random access to elements using their index.
 - **Operations:** Common operations include adding elements (`add`), retrieving elements by index (`get`), removing elements (`remove`), and checking size (`size`).
 - **Best Use Case:** When you need quick access to elements based on their position in the list.
- **Linked Lists:-**
 - **Structure:** Each element (called a "node") contains data and a pointer to the next node in the sequence.
 - **Node Class:** A basic linked list is often implemented with a "Node" class which holds data and a reference to the next node.
 - **Operations:** Insertion and deletion at the beginning or middle of the list are efficient.
 - **Best Use Case:** When frequent insertions or deletions are needed at various positions within the list.
- **Multi-dimensional Arrays:-**
 - **Concept:** An array that contains other arrays, creating a structure with multiple dimensions (like rows and columns in a matrix).
 - **Access:** Elements are accessed using multiple indices, one for each dimension.
 - **Example:** A 2D array can represent a table with rows and columns.
 - **Best Use Case:** When you need to store data in a grid-like format where each element can be accessed using multiple coordinates